

Image Extractor Agent

A semantic image-search engine. You describe a photo in plain English; the agent finds it inside a local vector database of image embeddings — **without ever reading the dataset's captions**. It understands the pictures directly.

3,000

IMAGES INDEXED

~40 ms

QUERY LATENCY

512-D

CLIP VECTORS

The problem

You have a folder of photos and only basic metadata — no descriptions, no labels. How do you find “a dog running on grass” among thousands of files? Filenames and EXIF data can't answer that. You need a system that understands the **visual content** of each image and matches it to a free-text query.

The solution

Encode every image once with **CLIP** into a 512-dimensional vector that captures its meaning. Encode the user's text query into the **same** space. The closest vectors (cosine similarity) are the best-matching photos. All retrieval runs **100% locally** — no cloud, no API key required.

🚫 Core rule — captions.txt is never used

Flickr8k ships with a `captions.txt` file of human descriptions. The whole point of this project is to prove the system understands images on its own, so that file is **excluded from indexing and retrieval entirely**. Search is driven purely by CLIP image↔text similarity. Captions used in the UI (optional) are generated by a vision model, not copied.

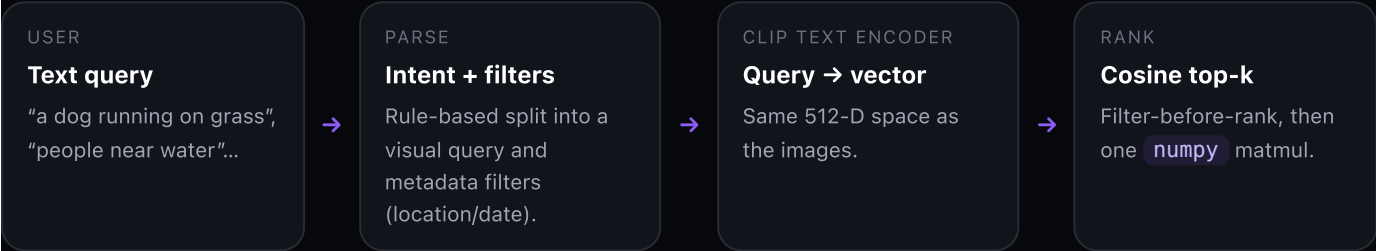
How it works — the pipeline

Two phases. The **index** is built once and cached; every **query** after that is a single fast matrix multiply.

① Offline indexing (once, ~5 min on CPU, zero API calls)



② Online query (per search, ~40 ms)



Why CLIP works across modalities

CLIP (Contrastive Language–Image Pre-training) is a **dual encoder**: an image encoder and a text encoder trained together on hundreds of millions of image–caption pairs so that a picture and its description land at nearly the **same point** in a shared vector space. Because both live in that one space, a text vector can be compared directly against image vectors. We use `ViT-B/32` pre-trained on `laion2b_s34b_b79k`.

Reading the score

Cross-modal cosine similarity is **low in absolute terms** — a strong match is typically **0.25–0.34**; you never see 0.9 across modalities. So the system **ranks** by score, it never thresholds. The UI maps the raw score to a friendly match %.

Key engineering decisions

DECISION	WHY IT MATTERS
Filter before ranking	For a query like “outdoor photos from July 2026”, we mask candidates by the date filter <i>first</i> , then rank only the survivors by CLIP similarity — so a metadata filter can never be starved out of the global top-k.
Stale-cache guard	On load, the index is validated against <code>metadata.csv</code> (same ids, same order, same model). A stale cache is rejected instead of silently returning wrong images.
Lazy, optional captions	Vision captioning is expensive. Search never calls OpenAI; captions are generated only on demand and cached — keeping search instant and token-free.
Pure image index	Only CLIP image vectors enter the index. Synthetic metadata has no visual grounding, so it influences results through filters, not through the embeddings.

Technology stack — in detail

Machine learning & retrieval

Component	Technology	Role
Embedding model	OpenCLIP ViT-B-32 / laion2b_s34b_b79k	Encodes images and text queries into one shared 512-D space. The heart of the search.
DL framework	PyTorch	Runs the CLIP forward pass (GPU if available, CPU otherwise).
Vector math	NumPy	Cosine similarity as a single normalized matmul over the cached index.
Atlas projection	scikit-learn (PCA + KMeans)	Projects 512-D → 2D and auto-clusters the whole dataset for the semantic map.
Dataset	Flickr8k via kagglehub	Deterministic 3,000-image subset; metadata generated with PIL + seeded RNG.
Vision captions (optional)	OpenAI gpt-4.1-mini	Generates captions/tags for explanations only — never for retrieval.

Backend

Component	Technology	Role
API server	FastAPI + Uvicorn	Endpoints: /api/search , /api/atlas , /api/samples , /api/health ; serves images.
Agent orchestration	Modular Python tools	parse_query · metadata_filter · search_images · explain_results · save_report — small composable tools, not one giant function.
Runtime	Anaconda (conda) env practical-ai-engineering , Python 3.13	Reproducible environment as required by the brief.

Frontend

Component	Technology	Role
Framework	React 18 + Vite 6	Fast SPA with hot-reload dev server and a proxy to the API.
Styling	Tailwind CSS v4	Utility-first styling; custom dark-neon design system with CSS variables.
Icons	Tabler Icons (webfont)	Consistent line-icon set throughout the UI.
Custom components	SVG match gauges, cursor spotlight, animated aurora + grain, glassmorphism, count-up stats, skeleton loaders, semantic Atlas scatter	A polished, “top-site” feel inspired by modern AI product design.

The product — semantic search

Type any description; get the top matching photos ranked by CLIP similarity, each with a match gauge, date, dimensions, the raw score, and a plain-language explanation of why it ranked. Search is instant because it is a single matrix multiply over the cached index.

Image Extractor Agent

semantic vision search

Search

Atlas

3,000 vectors

CLIP ViT-B/32 · fully local retrieval

Search photos in plain language

Describe what you remember. The agent embeds every image with CLIP and finds the closest matches in a local vector index — no captions, no cloud.

dog running on grass

Search

show

6

dog running on grass

people standing near water

a child playing outdoors

a person riding a bicycle

city street with people

a boat on the water

3,000

IMAGES INDEXED

512

VECTOR DIM

41 ms

QUERY LATENCY

Search for images of: dog running on grass

41 ms

#1



2026-08-14

500x334

0.337

Ranked 1 of 3000 candidates by CLIP visual similarity (score 0.3366); caption/tags unavailable (no OpenAI API key) — retrieval is pure CLIP image+text matching.

#2



2026-08-14

500x333

0.336

Ranked 2 of 3000 candidates by CLIP visual similarity (score 0.3355); caption/tags unavailable (no OpenAI API key) — retrieval is pure CLIP image+text matching.

#3



2026-07-02

500x420

0.335

Ranked 3 of 3000 candidates by CLIP visual similarity (score 0.3352); caption/tags unavailable (no OpenAI API key) — retrieval is pure CLIP image+text matching.

#4



2026-08-05

500x333

0.335

Ranked 4 of 3000 candidates by CLIP visual similarity (score 0.3351); caption/tags unavailable (no OpenAI API key) — retrieval is pure CLIP image+text matching.

#5



2026-07-29

500x454

0.334

Ranked 5 of 3000 candidates by CLIP visual similarity (score 0.3338); caption/tags unavailable (no OpenAI API key) — retrieval is pure CLIP image+text matching.

#6



2026-07-11

500x333

0.332

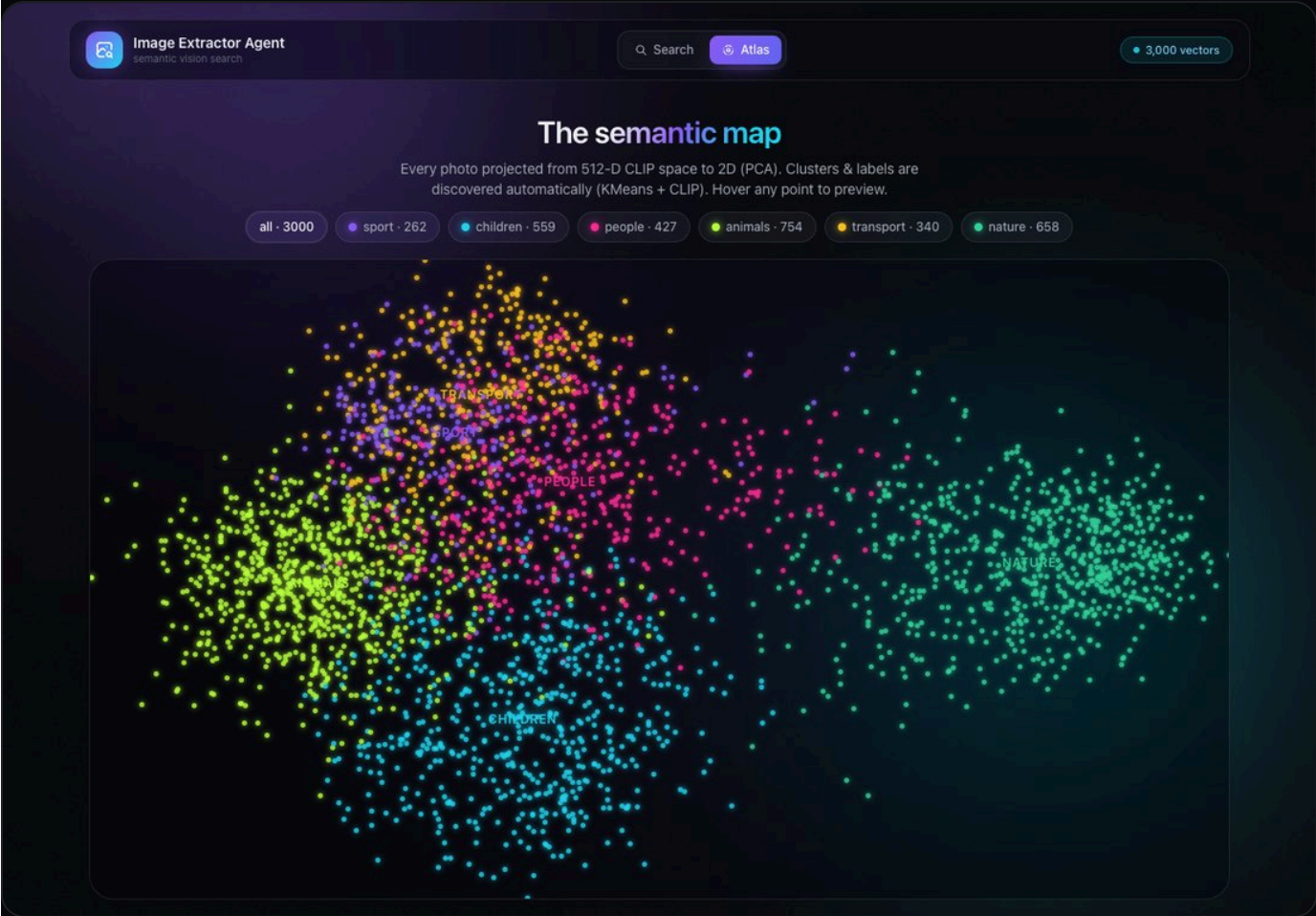
Ranked 6 of 3000 candidates by CLIP visual similarity (score 0.3319); caption/tags unavailable (no OpenAI API key) — retrieval is pure CLIP image+text matching.

100% local CLIP image+text retrieval · captions.txt never used for indexing · dates are synthetic · YDL 2026 · Day 5

Query "dog running on grass" → six correct matches in ~40 ms, scored 0.33–0.34.

The product — the semantic map (Atlas)

Every one of the 3,000 image vectors projected from 512-D CLIP space into 2D with PCA, then auto-clustered (KMeans) and labeled by the closest CLIP concept. Clearly separated neighborhoods — **nature**, **animals**, people, sport — show the embedding space really does organize images by meaning, with no supervision.



Running the project

```
conda activate practical-ai-engineering
pip install -r requirements.txt

# 1 · dataset: Flickr8k + deterministic 3,000-image subset
python scripts/prepare_flickr8k_subset.py --count 3000 --remove-raw

# 2 · frontend deps
cd frontend && npm install && cd ..

# 3 · build CLIP index (once) + start backend + frontend
./run_demo.sh          # → http://localhost:5173
```

`OPENAI_API_KEY` is optional — only for generated captions/explanations. Retrieval works fully offline. By default the backend does pure-CLIP search (zero API calls per query).

Project structure

```
image_extractor_agent/
├── scripts/   prepare_flickr8k_subset.py · build_index.py · make_notebook.py
├── src/       embedding_index · search_tools · vision_extractor · agent · report_writer · dataset_loader
├── backend/   main.py                    (FastAPI: search · atlas · samples · health)
├── frontend/  src/App.jsx · Atlas.jsx · ui.jsx    (React + Vite + Tailwind)
├── cache/     image_embeddings.npy · embeddings_meta.json
├── outputs/   search_results.json · search_report.md
└── image_extractor_agent_day5.ipynb    (required notebook demo)
```

Results & limitations

What works well

- Accurate top-k for concrete visual queries (dogs, water, bikes, crowds).
- Instant search after a one-time index build (~40 ms/query).
- Correct metadata filtering (filter-before-rank).
- Fully offline retrieval; no captions.txt, no cloud dependency.

Honest limitations

- Demo-scale dataset (3,000 of 8,091 images).
- ViT-B/32 misses fine detail (exact counts, text in images).
- Flickr8k has no real location metadata, so location is not used; only a **synthetic date** demonstrates filtering.
- Query parsing is rule-based; an LLM parser would handle richer phrasing.